

Emotion Recognition ToolKit (ERTK): Standardising Tools For Emotion Recognition Research

Aaron Keesing
University of Auckland
New Zealand
akee511@aucklanduni.ac.nz

Vithya Yogarajan
University of Auckland
New Zealand
vithya.yogarajan@auckland.ac.nz

Yun Sing Koh
University of Auckland
New Zealand
y.koh@auckland.ac.nz

Michael Witbrock
University of Auckland
New Zealand
m.witbrock@auckland.ac.nz

ABSTRACT

Many software packages and toolkits have been developed for machine learning, in particular for natural language processing and automatic speech recognition. However, there are few software packages designed for emotion recognition. Emotion datasets have diverse structures and annotations, and feature extractors often have different interfaces, which requires writing code specific to each interface. To improve the standardisation and reproducibility of emotion recognition research, we present the *Emotion Recognition ToolKit (ERTK)*, a Python library for emotion recognition. *ERTK* comprises processing scripts for emotion datasets, standard interfaces to feature extractors, and a framework for defining experiments with declarative configuration files. *ERTK* is modular and extensible, which allows for easily incorporating additional models and processors. The current version of *ERTK* focuses on emotional speech, however, the library is modular and can be easily extended to other modalities, which we plan for future releases. *ERTK* is open-source and available from GitHub: <https://github.com/Strong-AI-Lab/emotion>.

CCS CONCEPTS

• **Computing methodologies** → *Learning settings*; • **Software and its engineering** → **Software libraries and repositories**; • **General and reference** → *Experimentation*.

KEYWORDS

emotion recognition, toolkit, research software

ACM Reference Format:

Aaron Keesing, Yun Sing Koh, Vithya Yogarajan, and Michael Witbrock. 2023. Emotion Recognition ToolKit (ERTK): Standardising Tools For Emotion Recognition Research. In *Proceedings of the 31st ACM International Conference on Multimedia (MM '23)*, October 29–November 3, 2023, Ottawa, Canada.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MM '23, October 29–November 3, 2023, Ottawa, ON, Canada

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0108-5/23/10...\$15.00

<https://doi.org/10.1145/3581783.3613459>

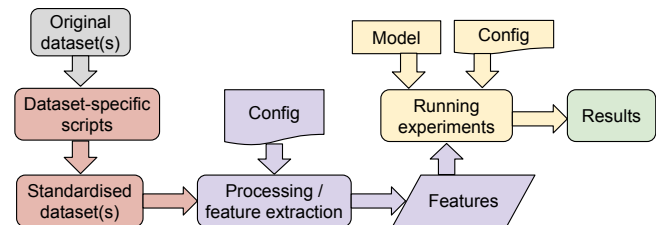


Figure 1: ERTK workflow. Download and standardise datasets, then extract features and run experiments with declarative configuration files.

ON, Canada. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3581783.3613459>

1 INTRODUCTION

As research in emotion recognition progresses, there is a need to standardise the tools and methodologies used for such research. This ensures consistency in the results obtained by different researchers and promotes reproducibility. Some attempts have been made to create standard *benchmarks* for emotion recognition, such as the *SERAB* benchmark [19], and the *audb* data format [22]. However, dataset processing, feature extraction, and running experiments are still specific, primarily to individual studies.

Several machine learning frameworks have been developed, including Scikit-learn [16], TensorFlow [1], and PyTorch [15]. These can be applied to many machine-learning tasks, but are too general to incorporate tools specific to emotion recognition. Dataset repositories such as *audb* [22] and HuggingFace *datasets*¹ have been developed to store and retrieve dataset metadata, but require dataset authors to voluntarily upload their data, which may conflict with copyright or licenses. Various tools have been developed specifically for emotion recognition, notably feature extractors such as OpenS-MILE [8], openXBOW [20] and DeepSpectrum [3]. However, these tools have different interfaces and require different configurations.

In this paper, we present *ERTK*, the *Emotion Recognition ToolKit*. This framework has been developed over the past three years to contribute to emotion recognition research methodologies, and the general workflow is shown in Figure 1. In particular, three main

¹<https://huggingface.co/datasets>

features of *ERTK* are (1) standardising emotion recognition datasets; (2) a uniform interface for feature extraction and instance processing; (3) a framework for declaratively specifying experiments with a configuration file. *ERTK* is modular and integrates with machine learning frameworks such as Scikit-learn [16], TensorFlow [1], and PyTorch [15]. Once a dataset is downloaded, a processing script can be used to convert the original dataset format to a standard format consisting of extracted annotations, a list of files in the dataset, and instances in any defined subsets of the data (e.g. training, validation and test splits). The instances in the data can then be processed with any of the built-in processors or third-party processors via a plugin interface. Finally, an experiment can be run according to a specification written in YAML², specifying the dataset(s) to load, model to use, training and validation setup.

ERTK integrates several existing tools with a uniform interface to simplify research workflows, and provides built-in processors and a plugin interface to integrate third-party tools easily. *ERTK* is open-source under an MIT license and is available from GitHub and PyPI³ for installation with Python’s pip installer. Full documentation is available at ReadTheDocs⁴. *ERTK* has already been used for research in emotion recognition [11, 12], and currently has 25 stars and 8 forks on GitHub. While the current focus of the toolkit is emotional speech, it can be extended to video and other modalities, which we plan for future releases.

2 DESIGN AND USE CASES

ERTK has three main components: dataset standardisation, instance processing, and declarative model training and evaluation. This section discusses these three components in detail and the motivation behind their design decisions. We then discuss additional utilities and tools that may be useful to researchers.

Throughout *ERTK*, we use structural configurations to specify parameters for models, datasets and experiments. Each configuration value has an associated type, and configurations can be nested. This allows a configuration to be validated for correctness and mapped to and from textual representations. *ERTK* uses YAML to represent configurations as human-readable text files and converts the textual representation to the corresponding structural configuration at runtime. Each structural configuration has an associated Python *dataclass* that represents the valid fields in the configuration. Fields may have a default value, in which case they need not be specified, while other fields require specifying a value.

ERTK has several command-line programs that provide an interface to its APIs for rapid iteration of experiments. Command-line programs exist for dataset processing, feature extraction and model evaluation. Configuration files can be specified on the command line, and specific parameters can be specified to override the parameters in the configuration file.

2.1 Dataset standardisation

Since many diverse emotion recognition datasets have varied file structures and annotation formats, one of the design goals for *ERTK*

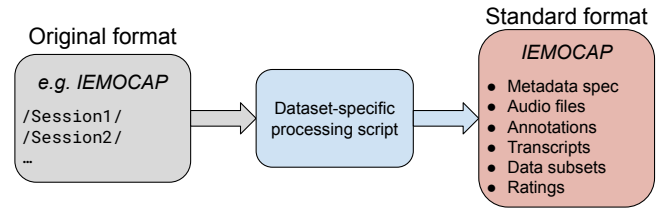


Figure 2: Dataset standardisation. A dataset-specific script converts the original downloaded dataset to a standard format.

is to have dataset-specific scripts that convert datasets into a standard format. Figure 2 shows the dataset standardisation process. A dataset-specific script converts the original downloaded dataset to a standard format. This format includes optional resampling and converting audio files to WAV, extracting CSVs for each annotation, and writing a list of files for each data subset. Currently, *ERTK* has scripts that process 35 emotion recognition datasets. Each dataset has metadata specified in a YAML configuration file due to its human readability and structure. This metadata file contains the dataset name, description, annotations, subsets, and the default location of extracted features. Using dataset-specific scripts avoids issues relating to copyright and licensing that may occur for publicly hosted repositories. This way, a user can download the original dataset, extract its contents, and then run the dataset-specific script to standardise the format.

Subsets of a dataset are given as text files containing file paths or instance names. The file paths specify the original audio files that can be used to load audio data for processing. There is always one subset named ‘all’ that includes all instances in the entire dataset. Other subsets may specify pre-defined training and validation sets, for example. A designated default subset is used when loading the data if no subset is specified as a parameter. The default subset is set to ‘all’ for many datasets, and for other datasets, the default subset may be the pre-defined training set. Specifying subsets allows loading only a portion of the data if the dataset is large.

Table 1: Example dataset annotations.

Annotation	Type
Emotion label	Categorical
Speaker	Categorical
Valence	Numeric
Transcripts	Text
Ratings	Multivalued

Table 1 exemplifies the types of annotations that may be present in a dataset. The values of annotations, particularly categorical annotations, can be used to partition the dataset (*i.e.* each instance has at most one value for a given annotation). Each annotation is stored as a CSV file with two columns: the instance name and the annotation value. Having each annotation separate allows additional annotations to be added later. *ERTK* normalises emotion labels to their English noun forms. This is to simplify combining

²<https://yaml.org/spec/1.2.2/>

³<https://pypi.org/project/ertk/>

⁴<https://ertk.readthedocs.io>

datasets for experiments since the same emotion categories can be combined by their label.

2.2 Instance processing

ERTK contains a framework for processing instances, such as feature extraction or transcription. This framework is designed to be modular and extensible; processors are installed as plugins stored in a global registry and loaded by name. In addition, processors have a standard function interface and use Python *iterators*. Hence, they can easily be chained together in a pipeline, lazily yielding one instance or batch at a time.

There are several processors built-in to *ERTK*, including:

- OpenSMILE [8] feature extraction
- OpenXBOW [20] vector quantisation
- Spectrogram extraction
- Audio resampling
- Encodec [6] embeddings extraction
- Processing with Fairseq [14] models
- Processing with Huggingface *Transformers* [23] models
- Processing with SpeechBrain[17] models
- Phonemisation of text using Phonemizer⁵
- Processing with Keras⁶ applications models
- VGGish and YAMNet [10] feature extraction

Each processor has configuration variables that modify its behaviour, which can be specified programmatically or using the `ertk-dataset process` command. This command takes additional arguments that specify configuration variables for the given processor. Any variables not given on the command line will use default values. Above, we specified that the `opensmile` processor should use the `eGeMAPS` config for OpenSMILE. The `process` command has an option for processing multiple instances in parallel if the processor supports it. In the above example, we can use an additional `--n_jobs 32` option to process 32 instances simultaneously if we have at least 32 CPU cores available. This dramatically speeds up processing for CPU-intensive processors such as OpenSMILE and spectrogram extraction.

Features can be stored in `netCDF` [18] format, but *ERTK* is also compatible with the `CSV` and `ARFF` (Weka [9]) formats, in addition to loading and saving raw audio. The `ertk-dataset process` command can also batch instances to take advantage of GPU memory and parallel processing speed. In the above example, we can use the `--batch_size 32` argument to transcribe 32 audio files simultaneously on the GPU if we have enough GPU memory.

2.2.1 Processor plugin interface. The instance processing framework is designed to be extensible so that third-party plugins can be installed easily. The built-in processors inherit from an abstract base class for processors, and optionally also the `AudioClipProcessor` class for audio files and `FeatureExtractor` for feature extractors. The base class stores a global registry of processors that can be retrieved by name, and processor plugins can be installed using the Python Entry points API under the `ertk.processors` group.

Table 2: Within-corpus results for six-class classification. Items in bold indicate the largest scores for each dataset. Mean fit time is given in seconds.

Dataset	Classifier	Features	UAR	Macro F1	Time (s)
CREMA-D	RBF SVM	eGeMAPS	54.0	53.3	138.0
		Wav2vec	66.3	66.1	75.1
	LR	eGeMAPS	52.7	51.9	8.9
		Wav2vec	66.5	66.2	12.3
	Aldeneh [2]	Mel-spec	52.3	50.0	74.0
	Zhao [24]	Mel-spec	58.8	58.7	142.1
EMO-DB	RBF SVM	eGeMAPS	66.1	63.6	1.4
		Wav2vec	85.5	83.8	2.9
	LR	eGeMAPS	63.9	61.5	1.0
		Wav2vec	86.7	84.9	3.3
	Aldeneh [2]	Mel-spec	62.1	59.6	17.9
	Zhao [24]	Mel-spec	44.7	32.3	55.0
RAVDESS	RBF SVM	eGeMAPS	48.9	47.7	5.4
		Wav2vec	66.1	64.9	12.9
	LR	eGeMAPS	50.3	49.3	1.1
		Wav2vec	68.8	68.0	9.4
	Aldeneh [2]	Mel-spec	53.0	51.6	26.3
	Zhao [24]	Mel-spec	52.5	51.7	103.3

2.3 Declarative experiments

The third main contribution of *ERTK* is the ability to run experiments from a declarative YAML configuration file. The experiment configuration specifies the datasets, features, and models to load and the training and evaluation configurations. The built-in command `ertk-cli exp2` can run experiments from the command line given a configuration file. This command specifies parameter overrides on the command line to enable quick iterating with different experiment parameters.

A critical aspect of the *ERTK* dataset API is that it can combine different datasets and modify annotations as necessary. Combined with the standard dataset format, this greatly eases training and evaluation on multiple datasets in cross-corpus or multi-corpus scenarios. Each annotation can be renamed or mapped to different values, which helps map emotion classes to binary arousal and valence. Portions of a dataset can be retained or dropped based on annotation values, for example, if only certain emotion classes are needed. Datasets are combined, assuming annotations are compatible with exceptions such as speaker IDs. After combining datasets, further manipulation can be performed as if the combined datasets were only one dataset.

The evaluation part of the configuration is specified as either cross-validation or pre-defined train, validation and test sets. Cross-validation can be specified overall instances or over groups of instances based on categorical annotations, such as speaker ID. This way, we can easily use speaker-independent cross-validation for evaluation. Pre-defined train, validation and test splits can be specified using the same selection criteria when loading datasets.

3 EXPERIMENTS

To demonstrate the utility of *ERTK*, we perform within-corpus, and cross-corpus benchmarks with common feature sets. We use the

⁵<https://github.com/bootphon/phonemizer>

⁶<https://keras.io/>

Table 3: Leave-one-corpus-out cross-corpus results for six-class classification. Items in bold indicate the largest values for each dataset. Fit times are given in seconds.

Test on	Classifier	Features	UAR	Macro F1	Time (s)
CREMA-D	RBF SVM	eGeMAPS	29.0	27.8	2.8
		Wav2vec	30.8	26.9	3.4
	LR	eGeMAPS	26.6	21.3	0.9
		Wav2vec	36.6	33.2	1.8
	Aldeneh [2]	Mel-spec	18.1	8.4	52.7
	Zhao [24]	Mel-spec	21.5	12.6	210.7
EMO-DB	RBF SVM	eGeMAPS	18.0	14.5	64.7
		Wav2vec	40.6	26.9	76.4
	LR	eGeMAPS	25.2	20.2	2.9
		Wav2vec	33.5	19.9	5.1
	Aldeneh [2]	Mel-spec	36.1	34.9	71.2
	Zhao [24]	Mel-spec	37.9	36.3	403.0
RAVDESS	RBF SVM	eGeMAPS	26.0	22.8	55.1
		Wav2vec	39.2	35.0	69.0
	LR	eGeMAPS	38.5	32.8	2.7
		Wav2vec	46.1	43.9	5.0
	Aldeneh [2]	Mel-spec	24.9	22.0	66.9
	Zhao [24]	Mel-spec	22.5	16.0	381.5

EMO-DB [4], CREMA-D [5] and RAVDESS [13] datasets in within-corpus and leave-one-corpus out evaluation. The within-corpus experiments are performed with leave-one-speaker-group out cross-validation, using ten speaker groups. We use eGeMAPS [7], and pre-trained Wav2vec [21] features, and we use support vector machine (SVM) and logistic regression (LR) as classifiers. We also compare against neural network models from Aldeneh and Mower Provost [2] and Zhao *et al.* [24] that use log-mel spectrograms as features. The within-corpus results are shown in Table 2 and the cross-corpus results in Table 3. Each of these experiments can be specified with an individual configuration file, or they can be specified with a common configuration file from which modifications are specified on the command line.

4 CONCLUSION AND FUTURE WORK

We have developed a Python library for emotion recognition, called the *Emotion Recognition ToolKit* (ERTK), to standardise emotion recognition research and facilitate reproducibility. In particular, ERTK provides three main features: (1) scripts to standardise existing emotion recognition datasets; (2) a standard interface to process instances and features; (3) a method for declaratively specifying experiments, including combining multiple datasets. We have given some examples of how to use each of these three features, both on the command line and programmatically.

While the current iteration of ERTK focuses on speech and audio, in future work we aim to integrate other modalities such as video. We also aim to integrate training pipelines for regression models and unsupervised learning. Finally, having additional interfaces to the increasing number of machine learning tools is vital for maintaining compatibility across projects.

REFERENCES

- [1] Martín Abadi, et al. 2016. TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems. <https://doi.org/10.48550/arXiv.1603.04467> [cs]
- [2] Zakaria Aldeneh and Emily Mower Provost. 2017. Using Regional Saliency for Speech Emotion Recognition. In *2017 IEEE International Conference on Acoustics, Speech and Signal Processing*. IEEE, New Orleans, LA, USA, 2741–2745. <https://doi.org/10.1109/ICASSP.2017.7952655>
- [3] Shahin Amiriparian, et al. 2017. Snore Sound Classification Using Image-Based Deep Spectrum Features. In *Interspeech 2017*. ISCA, Stockholm, Sweden, 3512–3516. <https://doi.org/10.21437/Interspeech.2017-434>
- [4] Felix Burkhardt, et al. 2005. A Database of German Emotional Speech. In *Interspeech 2005 - Eurospeech*. ISCA, Lisbon, Portugal, 1517–1520.
- [5] H. Cao, et al. 2014. CREMA-D: Crowd-Sourced Emotional Multimodal Actors Dataset. *IEEE Transactions on Affective Computing* 5, 4 (Oct. 2014), 377–390. <https://doi.org/10.1109/TAFFC.2014.2336244>
- [6] Alexandre Défossez, et al. 2022. High Fidelity Neural Audio Compression. <https://doi.org/10.48550/arXiv.2210.13438> [cs, eess, stat]
- [7] Florian Eyben, et al. 2016. The Geneva Minimalistic Acoustic Parameter Set (eGeMAPS) for Voice Research and Affective Computing. *IEEE Transactions on Affective Computing* 7, 2 (April 2016), 190–202. <https://doi.org/10.1109/TAFFC.2015.2457417>
- [8] Florian Eyben, et al. 2010. openSMILE: The Munich Versatile and Fast Open-Source Audio Feature Extractor. In *Proceedings of the 18th ACM International Conference on Multimedia*. ACM, Firenze, Italy, 1459–1462. <https://doi.org/10.1145/1873951.1874246>
- [9] Mark Hall, et al. 2009. The WEKA Data Mining Software: An Update. *ACM SIGKDD Explorations Newsletter* 11, 1 (Nov. 2009), 10–18. <https://doi.org/10.1145/1656274.1656278>
- [10] Shawn Hershey, et al. 2017. CNN Architectures for Large-Scale Audio Classification. In *2017 IEEE International Conference on Acoustics, Speech and Signal Processing*. IEEE, New Orleans, LA, USA, 131–135. <https://doi.org/10.1109/ICASSP.2017.7952132>
- [11] Aaron Keesing, et al. 2021. Acoustic Features and Neural Representations for Categorical Emotion Recognition from Speech. In *Interspeech 2021*. ISCA, Brno, Czechia, 3415–3419. <https://doi.org/10.21437/Interspeech.2021-2217>
- [12] Aaron Keesing, et al. 2020. Convolutional and Recurrent Neural Networks for Spoken Emotion Recognition. In *ALTA 2020*. ALTA, Virtual Workshop, 104–109.
- [13] Steven R. Livingstone and Frank A. Russo. 2018. The Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS): A Dynamic, Multimodal Set of Facial and Vocal Expressions in North American English. *PLOS ONE* 13, 5 (May 2018), e0196391. <https://doi.org/10.1371/journal.pone.0196391>
- [14] Myle Ott, et al. 2019. Fairseq: A Fast, Extensible Toolkit for Sequence Modeling. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (Demonstrations)*. Association for Computational Linguistics, Minneapolis, Minnesota, 48–53. <https://doi.org/10.18653/v1/N19-4009>
- [15] Adam Paszke, et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems*, Vol. 32. Curran Associates, Inc., Vancouver, Canada, 12.
- [16] Fabian Pedregosa, et al. 2011. Scikit-Learn: Machine Learning in Python. *Journal of Machine Learning Research* 12, 85 (2011), 2825–2830.
- [17] Mirco Ravanelli, et al. 2021. SpeechBrain: A General-Purpose Speech Toolkit. <https://doi.org/10.48550/arXiv.2106.04624> [cs, eess]
- [18] Russ Rew, et al. 1989. Unidata NetCDF. UCAR/NCAR - Unidata. <https://doi.org/10.5065/D6H70CW6>
- [19] Neil Scheidwasser-Clow, et al. 2022. SERAB: A Multi-Lingual Benchmark for Speech Emotion Recognition. In *2022 IEEE International Conference on Acoustics, Speech and Signal Processing*. IEEE, Singapore, 7697–7701. <https://doi.org/10.1109/ICASSP43922.2022.9747348>
- [20] Maximilian Schmitt and Björn Schuller. 2017. openXBOW – Introducing the Passau Open-Source Crossmodal Bag-of-Words Toolkit. *Journal of Machine Learning Research* 18, 96 (2017), 1–5.
- [21] Steffen Schneider, et al. 2019. Wav2vec: Unsupervised Pre-Training for Speech Recognition. In *Interspeech 2019*. ISCA, Graz, Austria, 3465–3469. <https://doi.org/10.21437/Interspeech.2019-1873>
- [22] Hagen Wierstorf, et al. 2023. Audb – Sharing and Versioning of Audio and Annotation Data in Python. <https://doi.org/10.48550/arXiv.2303.00645> [cs, eess]
- [23] Thomas Wolf, et al. 2020. Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, Online, 38–45. <https://doi.org/10.18653/v1/2020.emnlp-demos.6>
- [24] Z. Zhao, et al. 2019. Exploring Deep Spectrum Representations via Attention-Based Recurrent and Convolutional Neural Networks for Speech Emotion Recognition. *IEEE Access* 7 (2019), 97515–97525. <https://doi.org/10.1109/ACCESS.2019.2928625>